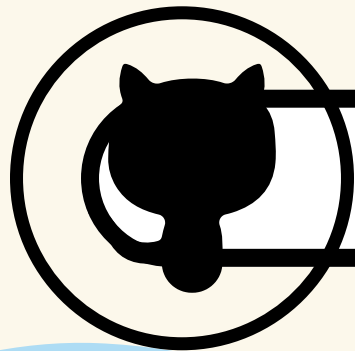


C++Template自主學習報告

STL學生管理系統



<https://github.com/aliu90473/Student-Management-System.git>

班級:資工一

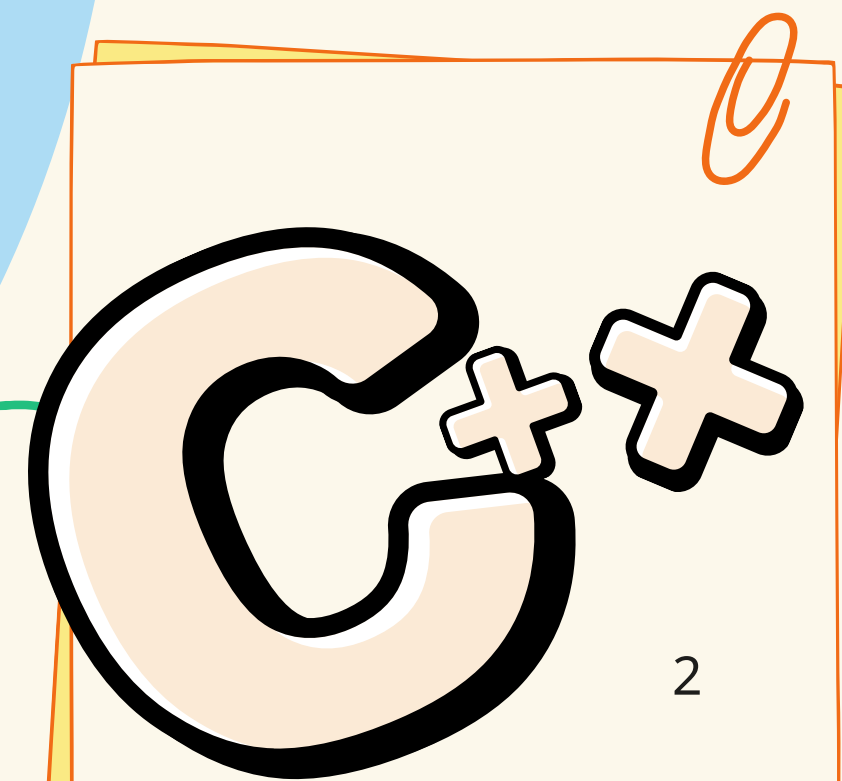
姓名:劉子亭

學號:S11459009

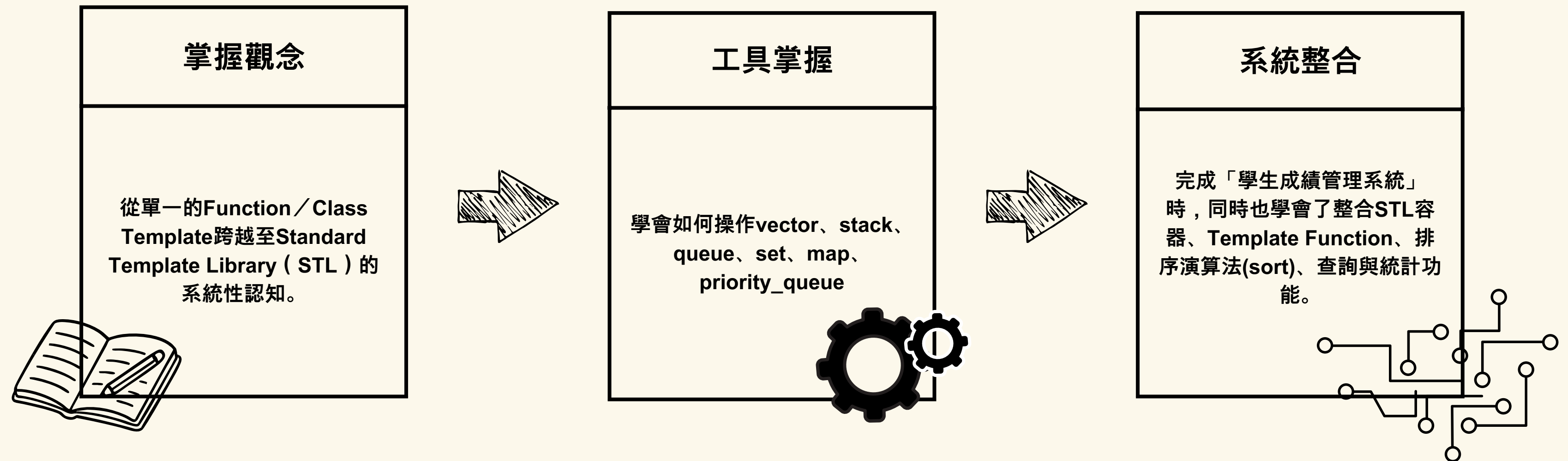
指導老師:林朝興老師

目錄

- ★ 自主學習主題與目標
- ★ Template 與 STL的關係
- ★ 常見STL容器用途整理
- ★ 第二週小專題設計說明
- ★ 主要程式架構與重要程式片段
- ★ 執行畫面或輸出結果
- ★ GitHub Repo Link 與檔案結構說明
- ★ 遇到的問題與解決方式
- ★ 學習心得
- ★ 未來可延伸方向

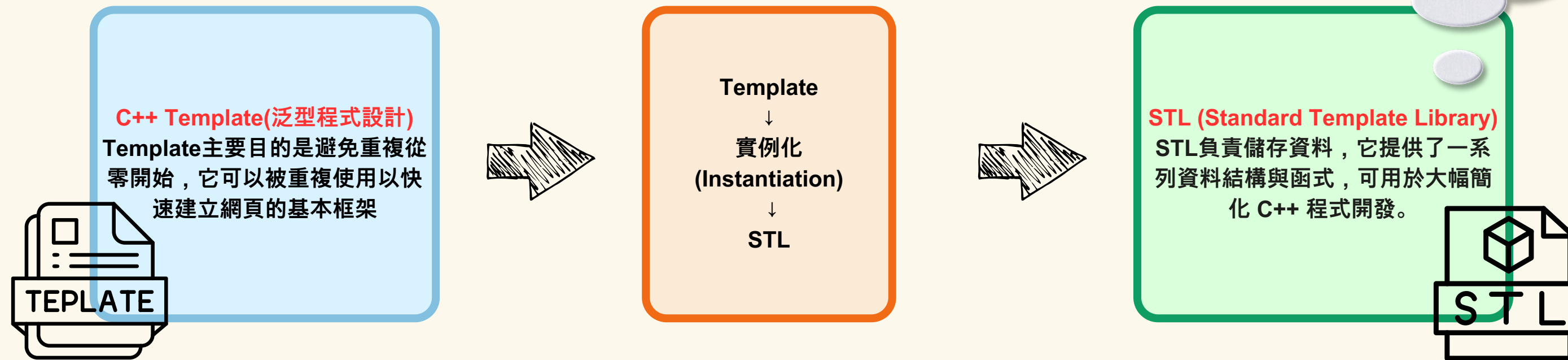


自主學習主題與目標



本次學習以C++ Template與STL為核心主題，透過實作學生成績管理系統，熟悉STL容器、演算法與Template Function的實際應用

Template 與 STL的關係



本專題中的 Template 與 STL 應用

```
// Template Functions
template <class T>
T getMax(T a, T b)
{
    return (a > b) ? a : b;
}

template <class T>
T getMin(T a, T b)
{
    return (a < b) ? a : b;
}
```

Template Function

本專題利用`getMax()`與`getMin()`完成最高分與最低分統計功能。不受資料型別限制提高程式碼重用性展現Template的優點

```
vector<Student> students;
```

```
sort(students.begin(), students.end(),
    [](const Student &a, const Student &b)
    {
        return a.score > b.score;
    });
```

STL Application

使用 `vector<Student>`儲存學生資料
使用`std::sort()`完成成績排序功能
STL提供高效且可靠的資料處理能力

Template是STL的核心基礎，而STL則是Template最成功且最廣泛的實際應用。
這次的自主學習透過`vector`、`sort`、`getMax()`與`getMin()`等功能，來結合Template與STL的整合運作方式。

STL是什麼??

- STL (Standard Template Library)
- STL 是 C++ 提供的標準模板函式庫 (Standard Template Library)。
- 建立於 Template (泛型程式設計) 技術之上。
- 提供許多已經實作完成的資料結構與演算法。
- 開發者不需要從零開始撰寫常用功能。
- 可以提高程式開發效率與程式碼重用性。
- 常見的STL應用:**vector/stack/queue/map/sort()/find()/count()**

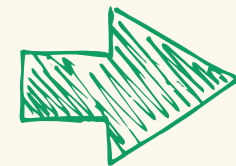
常見STL容器用途整理

容器名稱(Container)	存取特性(Key Trait)	專案適用場景(Use Case)	本專題中使用情形
vector	$O(1)$ 隨機存取	適合儲存大量資料，需經常瀏覽、插入與排序的場景。	本專題實際使用 <code>vector<Student></code> 儲存所有學生資料，並搭配 <code>std::sort()</code> 進行成績排序。
stack	狀態回溯	括號配對檢查 (Validating{[()]})、遞迴回呼、瀏覽器停靠。	第1週自主練習：括號配對檢查 (Validating{[()]})
queue	順序處理	模擬排隊服務櫃台系統、票務發放機制 (FIFO) 等。	第1週自主練習：排隊系統模擬 (Service Queue)
map	$O(\log N)$ 搜尋	以Key快速查找對應的Value，適合需要建立索引、快速查詢的場景。	可作為系統延伸：以學號 (Key) 快速反查學生資料與成績。

第二週小專題設計說明

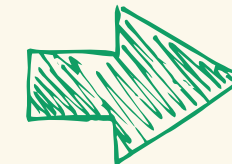
問題定義(Problem Definition)

設計一套簡易的學生成績管理系統，協管理學生的資料



系統目標(System Goal)

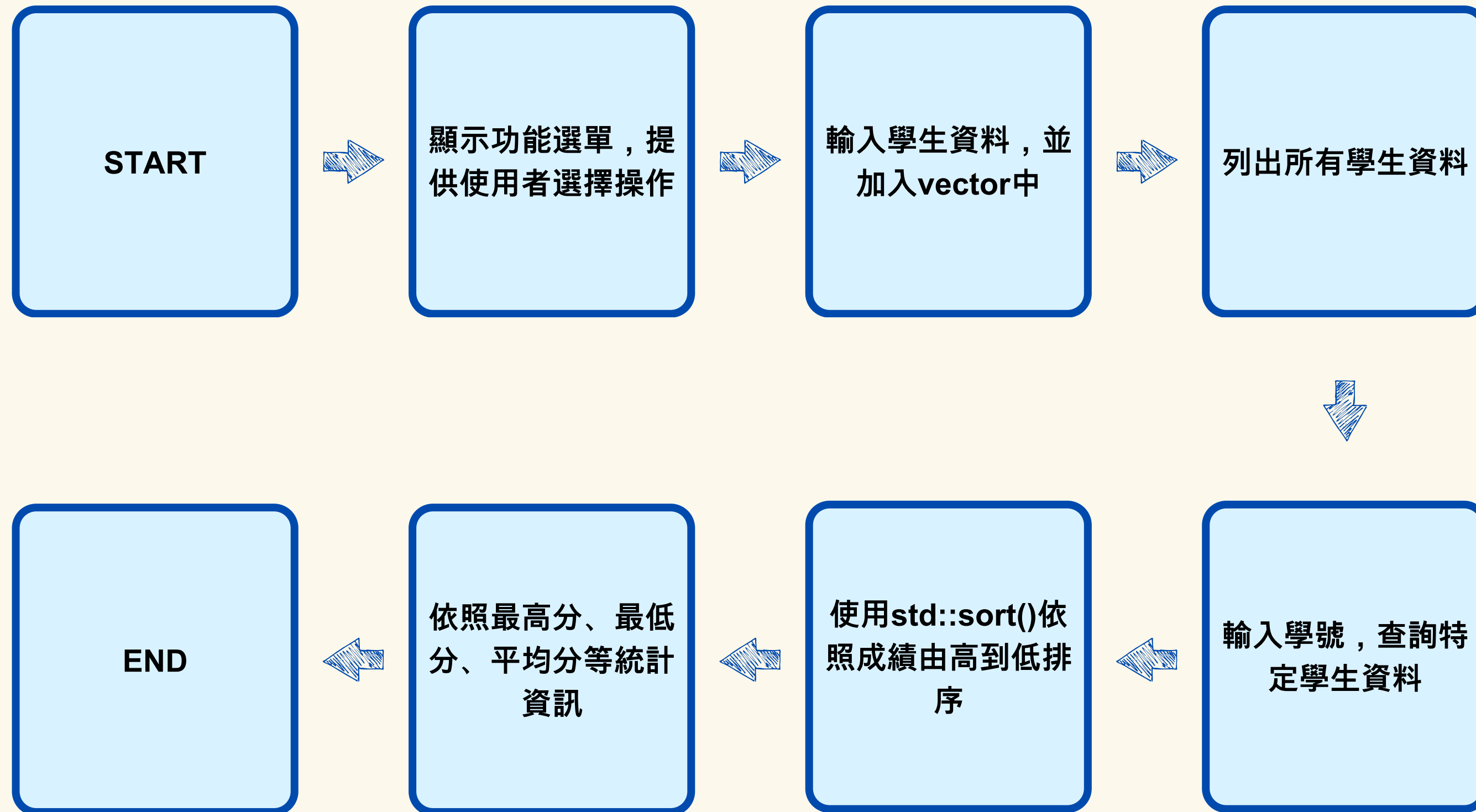
- 利用STL容器儲存學生資料，整合下列功能
 1. 新增資料
 2. 查詢資料
 3. 排序功能
 4. 成績統計
- 建立完整的學生管理流程



技術應用(Technologies Used)

- Struct：定義學生資料結構
- vector<Student>：儲存學生資料
- Template Function：
getMax()、getMin() 計算最高分與最低分
- std::sort()：進行成績排序
- C++17標準

第二週小專題系統流程圖



主要程式架構與重要程式片段--系統架構總覽

- 以vector<Student>作為主要資料容器，儲存所有學生資料。
- 利用STL演算法搭配Lambda Expression完成排序功能。
- 透過Template Function(getMax()與getMin())完成最高分與最低分計算。
- 整合新增、查詢、顯示、排序與統計等功能，提升程式效率與可維護性。

主要程式架構與重要程式片段--核心資料結構

`vector<Student>`（動態陣列）

優點（Pros）

- 動態陣列，記憶體連續，快取（Cache）效率佳。
- 可直接使用`std::sort()`搭配，能夠比較輕鬆完成成績排序。
- 實作簡單，程式可讀性高。
- 資料量適中時，整體效能表現良好。

缺點（Cons）

- 依學號搜尋需逐筆遍歷，增加時間複雜度。

主要程式架構與重要程式片段--核心演算法

C++17 Lambda
Expression
利用匿名函式定義比較邏
輯，
提升程式可讀性。

Iterator (迭代器)
指定排序範圍
對全部學生資料進行排
序。

```
sort(students.begin(), students.end(),  
    [](const Student &a, const Student &b)  
    {  
        return a.score > b.score;  
    });
```

自訂排序規則
以 score 作為比較依
據，
依成績由高到低排序。

主要程式架構與重要程式片段--最大值與最小值

```
// Template Functions
template <class T>
T getMax(T a, T b)
{
    return (a > b) ? a : b;
}

template <class T>
T getMin(T a, T b)
{
    return (a < b) ? a : b;
}
```

Template Function可套用於不同資料型別

- int
- double
- float
- char

```
for (const auto &s : students)
{
    highest = getMax(highest, s.score); //getmax
    lowest = getMin(lowest, s.score); //getmin

    total += s.score;

    if (s.score >= 60)
        pass++;
    else
        fail++;
}
```

vector<Student>逐一處理每一位學生資料。

動態更新最高分利用getMax()找到最高分

動態更新最低分利用getMin()找到最低分

執行畫面或輸出結果--新增與顯示學生資料

```
=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 1

===== Add Student =====
Enter ID: 1141
Enter Name: Apple
Enter Score (0-100): 60
Student added successfully!

=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 1

===== Add Student =====
Enter ID: 1142
Enter Name: Banana
Enter Score (0-100): 90
Student added successfully!

=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 1

===== Add Student =====
Enter ID: 1143
Enter Name: Papaya
Enter Score (0-100): 90
Student added successfully!

=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 2

===== Student List =====
ID          Name          Score
-----
1141        Apple          60
1142        Banana         90
Papaya      1143           90
=====
```

輸入內容

ID	Name	Score
1141	Apple	60
1142	Banana	90
1143	Papaya	90

系統功能

- 使用者輸入學號、姓名與成績。
- 資料加入 `vector<Student>`。
- 新增成功後顯示提示訊息。
- 輸出時依序顯示所有學生資料。

執行畫面或輸出結果--排序查詢學生資料

```
=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 3
```

Students sorted by score.

```
=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 2
```

===== Student List =====

ID	Name	Score
1142	Banana	90
Papaya	1143	90
1141	Apple	60

執行流程

Choose:3
↓
Students sorted by score
↓
Choose:2
↓
顯示排序後結果

排序結果

排序結果（由高到低）
Banana 90
Papaya 90
Apple 60

```
=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 4
```

Enter student ID: 1142

```
===== Student Found =====
ID   : 1142
Name : Banana
Score : 90
```

查詢條件

Enter student ID : 1142

查詢結果

ID:1142
Name:Banana
Score:90

執行畫面或輸出結果--統計功能

```
=====
Student Management System
=====
1. Add Student
2. List All Students
3. Sort By Score
4. Search By ID
5. Show Statistics
0. Exit
Choose: 5

===== Statistics =====
Average Score : 80.00
Highest Score : 90
Lowest Score  : 60
Pass Students : 3
Fail Students : 0
```

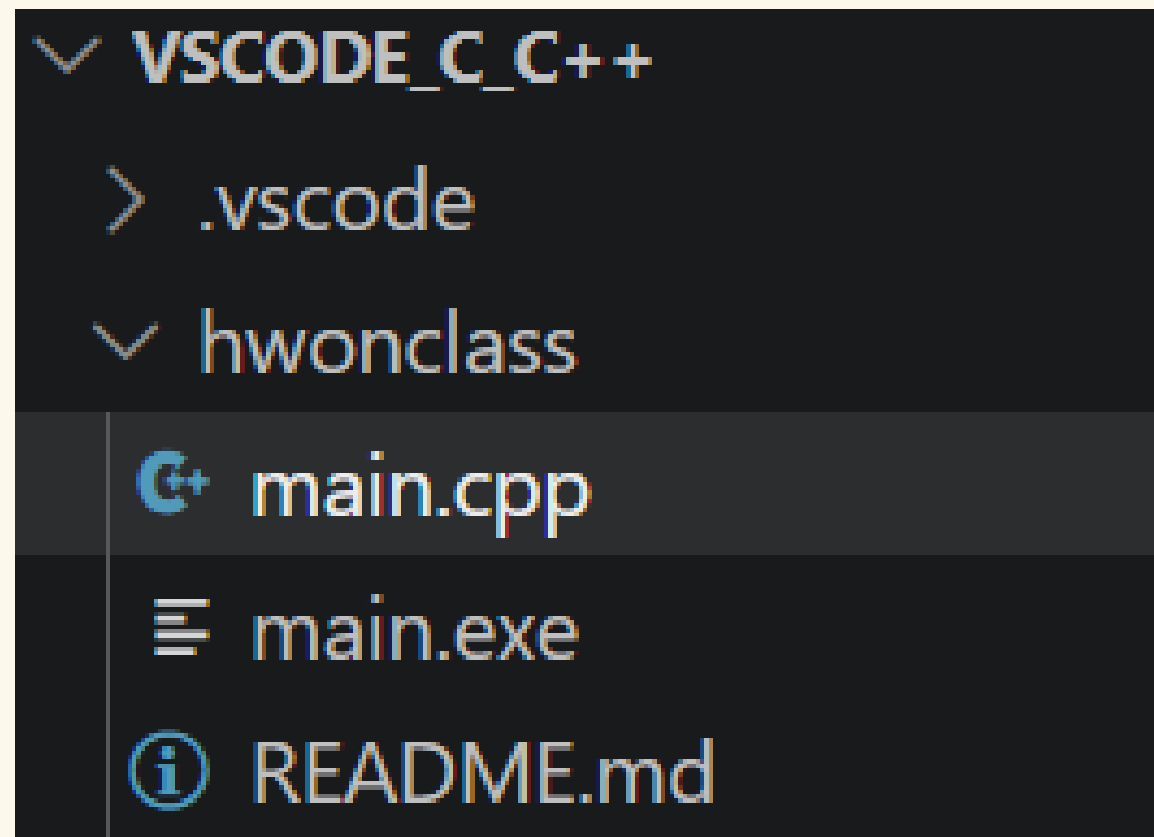
項目	數值
Average Score	80.00
Highest Score	90
Lowest Score	60
Pass Students (≥ 60)	3
Fail Students (< 60)	0

說明

- 利用 `getMax()` 求最高分。
- 利用 `getMin()` 求最低分。
- 計算平均分數。
- 統計及格與不及格人數。

GitHub Repo Link 與檔案結構說明

檔案結構(File Structure)



main.cpp

學生成績管理系統主程式

包含新增、查詢、排序、與統計功能

README.md

專案介紹以及程式編譯方式說明

編譯方式(Compilation)

```
g++ -std=c++17 main.cpp -o main  
./main
```

遇到的問題與解決方式

問題	原因分析	解決方法	程式顯示
問題 1：學號重複新增	- 未檢查學號是否已存在於 vector 中。 - 重複資料可能造成查詢錯誤。	新增資料前利用 for 迴圈逐一比對學號。 若發現重複則顯示「Error: Student ID already exists!」並停止新增。	<pre>for (const auto &st : students) { if (st.id == s.id) { cout << "Error: Student ID already exists!\n"; return; } }</pre>
問題 2：成績輸入範圍錯誤	- 使用者可能輸入不合理的成績。 - 若未檢查，可能造成統計結果錯誤。	限制成績範圍為 0 ~ 100。 若成績不在範圍內，顯示「Invalid score!」	<pre>if (s.score < 0 s.score > 100) { cout << "Invalid score!\n"; return; }</pre>
問題 3：查無學生資料	- 使用者可能輸入不存在的學號。 - 若未給予提示，使用者不清楚查詢結果。	搜尋完整個 vector 後，若未找到對應資料。 顯示「Student not found.」，提醒使用者查無資料。	<pre>cout << "Student not found.\n";</pre>

學習心得

在這次自主學習專題中，我透過實作學生成績管理系統，對C++ Template與STL有了更實際的認識。實際撰寫getMax()與getMin()等Template Function後，我慢慢理解到泛型程式設計的用途，也體會到利用Template可以減少重複程式碼，讓程式更容易維護與擴充。

除了Template之外，我也進一步接觸STL容器與演算法的應用。在本專題中，我使用vector作為主要資料儲存結構，並利用std::sort()搭配Lambda Expression完成成績排序功能。透過實際撰寫程式與測試，我了解到不同STL容器具有各自的特性，也開始思考為什麼這個專題適合使用 vector，而不是其他資料結構，讓我對STL的使用不再只是停留在課本上的內容。

在開發過程中，我也遇到不少問題，例如學號重複新增、成績輸入範圍檢查，以及查詢不到學生資料時如何提供適當提示等。剛開始常常只想到功能能不能完成，而忽略一些例外情況，後來透過不斷測試與修改，逐漸養成先思考資料正確性與使用者操作情況的習慣，也讓我了解到程式除了完成需求之外，穩定性與可靠性同樣重要。

完成本專題後，我不僅更加熟悉STL容器與Template Function的使用方式，也對程式架構、模組化設計以及資料結構的應用有更深入的理解。雖然目前完成的是一個簡單的系統，但也讓我對之後的資料結構與演算法課程更有興趣，希望未來能加入更多功能，例如資料修改、刪除以及檔案存取功能，讓整個系統更加完整。

未來可延伸方向

雖然目前完成的學生成績管理系統已經具備新增、查詢、排序與統計等基本功能，但我覺得還有許多地方可以繼續改進與擴充。

首先，希望之後能加入資料儲存功能，讓學生資料可以儲存在檔案中，而不是每次重新執行程式後都要重新輸入資料。這樣除了能節省時間，也能讓整個系統更接近實際應用。

另外，目前系統只能利用學號查詢學生資料，未來希望能增加更多功能，例如依照姓名搜尋、修改學生資料、刪除學生資料，以及顯示成績前幾名等功能，讓系統能夠處理更多需求，也提高使用上的便利性。

在這次專題中，我是利用VSCode的Terminal進行操作。雖然功能都能正常執行，但畫面看起來比較簡單。未來如果有機會，希望能學習更多程式設計相關知識，嘗試製作較美觀的操作介面，讓使用者能更容易使用整個系統。

透過這次自主學習，我發現程式設計不只是把功能完成而已，還需要考慮資料管理、程式架構以及使用者操作的方便性。雖然目前完成的系統還有很多不足，但也讓我對C++、STL以及資料結構有更進一步的認識，希望未來能持續學習，慢慢把這個系統做得更完整。

THANKS

